

Understanding Raw LLMs vs AI Agents: A Forward-Deployed Engineer's View

Written by Shaan Goswami

I recently had a chat with a friend who's excited about AI/ML. He confidently claimed that "raw LLMs always give correct answers" and can remember anything I just told them. I pushed back: no, plain LLMs are just prediction engines – they lack true memory or reasoning. For example, a vanilla GPT-4 (or Claude) can't reliably multiply large numbers or remember your name across sessions, because it's stateless and bound by its context window. In practice, systems like ChatGPT and Claude aren't just bare LLMs – they're agents built on top of LLMs, with added memory, tools, and control loops that make them look far smarter.

In this post I'll explain the key differences. We'll see why a raw transformer is "tremendous power sitting on a crate", why it can hallucinate or forget things, and how modern AI products wrap the LLM "engine" in a car with brakes, navigation, and a memory system.

The Myth of the Omniscient LLM

First, recall how a transformer LLM works. It predicts the next token based on its training data, using an attention mechanism over a limited context window. Each token attends to all others, creating n^2 pairwise relationships. As this window grows, early context gets "stretched thin" and the model becomes less precise at retrieving older information. In other words, LLMs have a finite "attention budget". If you tell the model something, then proceed with a long conversation, the earliest tokens may effectively fade from memory unless explicitly managed.

Moreover, LLMs are stateless by design. They process each input and output in isolation. As Lakshmi Narayanan (who worked on AI at OpenAI) noted: "LLMs are stateless. Just like a calculator: you give an input, it gives an output, then it forgets everything." ChatGPT, on the other hand, adds layers on top: a conversation UI, session context, and optional memory features. In his words: "LLM = Engine – ChatGPT = Engine + Car + Safety + Memory + UI."

In practice this means: a raw LLM (like an API call to GPT-3 or GPT-4 without any extra system) has no persistence beyond the immediate request. If you ask it a question, it uses just the provided prompt tokens. Once the call ends, the model doesn't "remember" anything in its weights. In contrast, products like ChatGPT use the chat history (or even a long-term memory layer) to supply relevant tokens for future replies.

This statelessness also shows up in performance. For example, state-of-the-art models still struggle with simple arithmetic. A recent paper found that even GPT-4 and Llama-3.2 (90B) fail at 4×4-digit multiplication. In short: LLMs don't "calculate" like humans – they pattern-match tokens. Without special training tricks or tools, they will hallucinate or guess numeric answers.

Key points about raw LLM limits:

- **Finite context:** Long dialogues or documents can overwhelm attention. The model might lose track of earlier facts (like your name) if they fall outside the window.
- **Statelessness:** After a chat ends, a raw API call has no memory of past chats. (Chat platforms simulate memory by feeding history back into prompts or using external memory.)
- **No external actions:** By itself, an LLM can't browse the web, run code, open files, or do math. It only "speaks" text based on training.
- **Probabilistic answers:** It's essentially a very smart autocomplete, not a database of truth. This leads to confident-sounding but incorrect "hallucinations."

In other words, a vanilla LLM is like "an engine on a crate" – it's powerful but purposeless without scaffolding.

The Agent Harness: Engine + Tools + Memory

Real-world AI assistants (ChatGPT, Claude, etc.) wrap that raw engine in a harness of code and logic that gives it capabilities. As Barath Kumar Padmanaban described when building an AI "harness":

"A raw LLM is shockingly limited. It can't open a file. It can't run a command. It can't even remember what it said thirty seconds ago – it's stateless. All it does is read some text and predict the next chunk of text... The harness is the boring, unmagical code that gives the engine hands."

In practice, an AI agent consists of two layers:

The LLM (reasoning engine): This is the same transformer model (GPT, Claude, Gemini, etc.) that generates text. It's extremely powerful at pattern recognition and language.

The Harness (agent logic): Custom software that prepares prompts, invokes tools, manages context/memory, and processes outputs. This is where most engineering happens.

This harness typically implements an agent loop:

1. **Receive user query:** The user's request (plus system instructions) is assembled into the prompt.
2. **Reason:** Call the LLM on this prompt. The model may generate a final answer or, more often, an intermediate step like "Let me check X" or a tool request.
3. **Act:** If the LLM's output calls a tool (e.g. "search Google for...", "run this Python code," "query the database"), the harness executes that tool in the real world.
4. **Observe:** The tool's result is taken as new information, and the harness appends it back into the conversation context.
5. **Repeat:** Go back to reasoning with the updated context. Continue looping until the agent decides the task is done.

This is exactly the pattern behind coding assistants like Claude Code or ChatGPT with plugins. In Barath's simple example, the code loop looked like:

```
const msg = await callModel(messages);
if (msg.tool_calls) {
  for (const call of msg.tool_calls) {
    const result = await runTool(call.name, call.args);
    messages.push({ role: "tool", content: result });
  }
}
// loop back and call model again
```

Each iteration is one “agent turn”: the model suggests an action, the system executes it, and feeds the result back. The model never directly manipulates the outside world – it asks the harness (the “hands”) to do so.

In summary, a modern AI agent is far more than a single API call. It combines: system prompts, user instructions, a call loop, external tools (search, code execution, browsers, calculators, etc.), and sometimes memory storage. Pete Hodgson’s analysis of Claude Code emphasizes this: every coding agent has a detailed system prompt, memory or rule files in the codebase, and a set of tools (read/write files, run shell commands, web search, etc.). These determine what the LLM can do and how it behaves.

In effect:

- **System prompt:** Guides the LLM’s role (“You are an expert coder” or “You are a helpful assistant”).
- **Memory/Context:** Extra info (previous chats, user profile, project files) that’s inserted into the prompt.
- **Tools:** Functions like `search()`, `calculate()`, `run_code()`. The LLM outputs a tool-call; the harness runs it and returns the answer as context.
- **Loop controller:** Logic to keep the conversation going until completion.

All of this engineering lives in the harness, not the model weights. This distinction helps explain why two agents using the same base model can behave very differently. It’s the harness (prompts + tools + context management) that makes Claude Code faster at coding than, say, ChatGPT without plugins.

Memory & Context: Beyond the Model’s Window

Because raw LLMs forget once the prompt ends, agents build memory systems. Think of memory as any way the agent stores facts outside the immediate token window. Two common patterns are:

Session context: For a single conversation, the system may append all past dialogue into each prompt (up to a limit) or summarize older turns. This is how ChatGPT remembers what you said earlier within the same chat. In fact, the ChatGPT UI hides this: you think of it as “just talking,” but under the hood your entire chat history is being fed back into the model (or summarized) to preserve continuity. OpenAI’s Agents SDK even provides a Session object to automatically manage this history, trimming or compressing it as needed.

Persistent memory: Some assistants store information across chats. For example, ChatGPT’s “memory” feature can remember your name, preferences, or project details over time. Anthropic similarly launched a memory tool where Claude can read/write notes to a file. These memories are external to the model’s

weights. They might live in a database or file system, and the agent retrieves them (via retrieval augmentation) to insert into the prompt when relevant.

Anthropic's example of Claude playing Pokémon illustrates the power of agentic memory. As Claude navigated thousands of game steps, it wrote down strategic notes ("in Route 1, Pikachu has gained 8 levels...") to a memory file. When the session restarted, it reloaded those notes and continued. This let the agent remember game state and long-term objectives far beyond the transformer's context window – something no raw LLM could do on its own. In effect, Claude treated its own "notes" as memory, enabling hours-long planning and learning in the game.

In practical terms, what does this mean? If you tell ChatGPT "remember that I'm a vegetarian," it will save that fact and apply it to future chats. If you have Claude Code browsing a codebase, it can create a `NOTES.md` or use a built-in memory API to recall earlier findings. By contrast, a plain GPT-4 API call would forget it immediately.

Context management: Agents also proactively manage the content they keep. They may trim irrelevant parts of the conversation or summarize old threads to a few key points. This ensures the "attention budget" isn't wasted on stale details. OpenAI emphasizes that even GPT-5's enormous 272k-token window can be overwhelmed by uncurated histories, so smart agents trim and summarize to keep only the high-signal tokens.

Tools & Reasoning: How AI Agents Do Math (and More)

So far we've focused on memory and prompts. Now consider functional limitations: a raw LLM can't open a calculator or web search. Agents overcome this by calling tools.

Take math as an example. Standard GPT models are surprisingly bad at arithmetic because they don't actually compute – they just output what looks statistically plausible. However, OpenAI's productized ChatGPT can do complex math by using a Python interpreter (the "Code Interpreter" or Advanced Data Analysis tool). Internally, if you ask ChatGPT to calculate a large multiplication, it can write a small Python script and execute it safely. As the docs say, the Code Interpreter "allows models to write and run Python code in a sandboxed environment to solve complex problems in domains like ... math." In effect, the agent says to itself: "I need exact math; let me call Python."

Anthropic's Claude likewise can call tools. In the latest releases, Claude can do extended thinking with plugins like web search or file manipulation. For example, with a connected browser or search tool, Claude can fetch factual data or real-time info when answering. Or if integrated in an IDE, Claude Code can run tests and commands via a secure shell tool.

Key examples of tool-use vs raw LLM failure:

- **Arithmetic:** Ask raw GPT-4, "What is 12345×6789 ?" and it will likely err. But ChatGPT with Code Interpreter or the Wolfram plugin returns the exact product. Tools (Python, WolframAlpha) do the arithmetic.

- **Fact-checking:** A training-limited model might hallucinate a date or statistic. An agent can search the web (or query a knowledge base) to verify facts. For instance, ChatGPT with the Bing or web-search plugin will actually fetch up-to-date info instead of guessing.
- **File operations:** Claude Code can open your code files, run grep, execute shells, etc. A raw LLM can't even access your disk.

In short, agents let the LLM plan and call out to external calculators, databases, or APIs. The LLM might reason (e.g. chain-of-thought) to determine which tool to use next, but the heavy lifting is done by specialized functions. The transformer stays in its lane (text reasoning) while delegating actions to the harness. This architecture – a loop of “think, act, observe” – is central to agentic AI.

ChatGPT, Claude, and the Full Stack of AI

Bringing it together, why do ChatGPT and Claude appear so “intelligent” that my friend assumed they were just omnipotent LLMs? It's because under the hood they're complete AI agent platforms.

ChatGPT (OpenAI) wraps GPT models in a polished chat interface with short-term memory (the current conversation) and optional long-term memory. It also offers plugins (like Code Interpreter, web search, etc.) that let the model execute code or browse the web on demand. This means ChatGPT effectively becomes: model + memory + computation. As Lakshmi put it: “Engine + Car + Safety + Memory + UI.”

Claude (Anthropic) adds its own twist: clear system prompts (“you are the assistant”), semantic search over your docs, and an API with tools (browser, calculator, file system). The newest Claude (Opus 4) can even create its own memory files and manage them. So when you chat with Claude, it can learn from the conversation and store relevant info.

Neither product is just “GPT-4 in a new outfit.” They are engine plus harness. That's why they can do ongoing reasoning and “remember” you: the continuity comes from software, not from miracle neurons.

For example, you might find ChatGPT greeting you by name in a new chat. That's because its memory feature saved your name. In contrast, if you called the raw GPT-4 API twice with “Hi, my name is Alice. ... Hello?” in separate calls, it would have no idea who Alice is on the second call – no memory in the model's weights. Similarly, Claude might recall project details because the developer enabled its memory file store.

Conclusion: Crafting the Future of AI Agents

In summary, raw LLMs are powerful but incomplete engines. They excel at language prediction, but alone they can't maintain long-term plans, perform exact calculations, or interact with the world. True agents (like ChatGPT, Claude Code, etc.) are LLMs embedded in an ecosystem: detailed prompts, loop logic, tools, and memory layers.

As a forward-deployed engineer, my job is often to build that harness. I choose which tools to expose, how to format prompts, and how to manage context. The rewards are huge: with even a modest loop, we turn a “static” model into an autonomous problem-solver. But it means acknowledging: the intelligence we see is partly in the design around the model, not just the model itself.

So, next time someone brags that “the LLM got it right,” remind them of the untold scaffolding behind the scenes. And remember, a brilliant human “engineer” (and the right tools) can make the LLM soar.

— Shaan Goswami